

SOFTWARE CHANGE IMPLEMENTATION REPORT

Course Management System

Change Request: CR-005

Global Session Expiry and Friendly Re-Login

MH Anush

Student ID: 24ICT16

Software Configuration Management - SCM (K72C001M12)

Version 1.0

06 October 2026

University College of Jaffna / Department of Interdisciplinary Studies

Table of Contents

1. Change Request	1
1.1 Change Request Identification	1
1.2 Business Requirement	1
1.3 Existing Behaviour	1
1.4 Requested Change	1
1.5 Change Objective	1
2. Change Analysis & Impact	2
2.1 Change Scope	2
2.2 Assumptions and Constraints	2
2.3 Impact Analysis	2
2.4 Affected Components and Files	2
3. Requirements & Acceptance Criteria	3
3.1 Functional Requirements	3
3.2 Non-Functional Requirements	3
3.3 Security Requirements	3

3.4 Acceptance Criteria	3
4. Risk & Configuration Management	4
4.1 Risk Analysis	4
4.2 Risk Mitigation Summary	4
4.3 Configuration Items	4
4.4 Baseline and Version Information	5
4.5 Commit Information	5
5. Implementation	5
5.1 Implementation Summary	5
5.2 Technical Implementation	5
5.3 Files Modified	6
5.4 Database Changes	6
5.5 API Changes	6
5.6 Authentication / Authorization / Security Changes	7
5.7 Implementation Logic	7
5.8 Error and Data Handling	7

5.9 Important Technical Decisions	8
6. Testing & Verification	8
6.1 Testing Strategy	8
6.2 Testing Environment	9
6.3 Test Data / Test Account	9
6.4 Test Cases	9
6.5 Test Summary	10
6.6 Regression Verification	10
6.7 Acceptance Criteria Verification	10
7. Traceability & Evidence	11
7.1 Change Traceability	11
7.2 Embedded Evidence	11
8. Version Control & Change Control	12
8.1 Repository and Branch Information	12
8.2 Code Review / Change Review	12
8.3 Change Control Outcome	12

9. Final Status & Closure	13
9.1 Final Change Status	13
9.2 Change Scope Summary	13
9.3 Final Summary	13
9.4 Outstanding Issues / Limitations	13
9.5 Follow-up Actions	14
9.6 Closure Checklist	14
9.7 Closure Comments	14
9.8 Final Closure	14
Appendix A – Additional Evidence	15
Evidence A-01	15
Evidence A-02	15
Appendix B – Additional Test Evidence	16
Test Evidence B-01	16
Test Evidence B-02	16
Appendix C – Original Change Request Reference	16

1. Change Request

1.1 Change Request Identification

Field	Details
Change Request ID	CR-005
Change Request Title	Global Session Expiry and Friendly Re-Login
Priority	High
Difficulty	Easy
Change Type	Enhancement / Security / Frontend Authentication Change
Implementation Status	Completed
Student / Developer	MH Anush (Student ID: 24ICT16)
Date	06 October 2026

1.2 Business Requirement

Users must receive a clear and user-friendly message when their authentication session expires or becomes invalid instead of seeing confusing API errors.

The frontend must provide global handling for HTTP 401 Unauthorized responses using an Axios response interceptor. When an authenticated user's session expires, the application must safely clear the stored authentication information, redirect the user to the Login page, and inform them that their session has expired.

The page the user was attempting to access must be preserved so that, after successful re-authentication, the user can be returned to the original page.

The implementation must distinguish between expired sessions, invalid login credentials, network failures, and access-denied responses.

1.3 Existing Behaviour

Before implementing this change:

- JWT authentication tokens expire after a fixed period on the backend.
- The frontend did not have a global Axios response interceptor for authentication failures.
- Expired or invalid tokens remained in `localStorage`, causing subsequent requests to fail with broken views or raw alerts.
- There was no consistent global redirect to the Login page when a session expired.
- The Login page did not display a dedicated session-expired message.
- The originally requested page was not preserved when users were redirected to Login.
- Multiple simultaneous API requests produced multiple 401 responses, causing flickering and repeated redirects.
- Network errors were not cleanly isolated from authentication failures.
- Accessing wrong-role pages silently navigated to fallback routes without informative feedback.

1.4 Requested Change

Improve frontend authentication handling by implementing global session-expiry detection and a friendly re-login flow:

- Add a global Axios response interceptor in `api.js`.
- Detect HTTP 401 Unauthorized responses on authenticated requests.
- Exclude the Login request (`/auth/login`) from session-expiry handling.
- Clear stored authentication token (`token`) and user information (`user`) from `localStorage`.
- Redirect the user to `/login` and preserve the target destination route.
- Display exact message on the Login page: **"Your session has expired. Please log in again."**
- Continue displaying **"Invalid username or password"** for failed login credential attempts.
- Prevent duplicate redirects caused by multiple simultaneous 401 responses using a concurrency guard.
- Ensure network errors do not clear authentication information or trigger logouts.
- Ensure HTTP 403 Forbidden responses do not log out users; display **"Access denied. You do not have permission to access this page."**
- Return the user to the originally requested page upon successful re-login.

1.5 Change Objective

The objective is to eliminate user confusion during JWT session expiration by establishing automated client-side interception, secure credential cleanup, friendly status notices, and frictionless route restoration.

2. Change Analysis & Impact

2.1 Change Scope

In Scope:

- Global Axios response interceptor configuration in `frontend/src/services/api.js`.
- Exclusion filter for `/auth/login` requests to prevent infinite loops.
- Authentication state clearing using `clearAuth()` in `auth.js`.
- Target destination preservation via React Router state and `sessionStorage`.
- Session expired warning banner on `Login.jsx`.
- Invalid login credentials error handling separation.
- Concurrency guard to collapse simultaneous 401 errors into a single redirect.
- Network failure isolation ensuring credentials remain intact during connectivity loss.
- Access Denied UI rendering in `ProtectedRoute.jsx` for unauthorized roles while maintaining active session.
- Post-login redirection back to the preserved page.

Out of Scope:

- Backend JWT generation, signing key, or expiration timeout modifications.
- Backend database schema or user record alterations.
- Introduction of new authentication APIs or session management endpoints.

2.2 Assumptions and Constraints

Assumptions:

- Existing backend API issues HTTP 401 for expired tokens and HTTP 403 for unauthorized roles.
- Existing `localStorage` keys (`token`, `user`) remain canonical.
- Client browser supports standard `sessionStorage` and custom DOM events.

Constraints:

- Strict frontend-only change; zero backend routes, controllers, or database modifications.
- Zero extra API requests solely for session verification.
- Login requests must never trigger session-expiry handling.
- Network failures must never log out users.

2.3 Impact Analysis

Identify the areas that could be affected by the change.

Area	Impact	Details
Frontend	Medium	Axios interceptor, Login page, ProtectedRoute, and App routing updated.
Backend	None	No backend endpoints or controller logic modified.
Database	None	No schema or data modifications required.
API	None	Existing REST API endpoints and contracts used without change.
Authentication / Authorization	High	Centralized 401 handling, session clearing, and role boundary protection implemented.
Security	High	Purging stale/expired tokens immediately prevents token reuse vulnerabilities.
User Interface / Styling	Low	Added session-expired amber banner and Access Denied card.
Configuration	None	No environment variables or build configurations altered.
External Services	None	No external dependencies impacted.
Performance	Low	Interceptor executes synchronously in <1ms per request with zero overhead.
Testing	Medium	Validation across all authentication, concurrency, network, and role states.
Documentation	Low	Implementation report authored for change control compliance.

Area	Impact	Details
Other	None	None.

2.4 Affected Components and Files

Record the components and files actually affected by the implementation.

Component / File	Type	Changed?	Purpose / Reason for Change
frontend/src/services/api.js	Frontend / Service	Yes	Configured global response interceptor for 401 handling, login exclusion, and duplicate redirect guard.
frontend/src/pages/Login.jsx	Frontend / Page	Yes	Added session-expired alert rendering, preserved destination redirection, and credentials failure separation.
frontend/src/components/ProtectedRoute.jsx	Frontend / Component	Yes	Added Access Denied layout for unauthorized roles while maintaining authenticated session.
frontend/src/App.jsx	Frontend / Router	Yes	Integrated <code>SessionExpiredListener</code> for smooth in-app navigation via React Router.

3. Requirements & Acceptance Criteria

Requirements must be derived from the original Change Request.

3.1 Functional Requirements

ID	Requirement	Status
FR-01	Implement global Axios response interceptor for authentication responses.	Met
FR-02	Detect HTTP 401 Unauthorized responses globally on authenticated requests.	Met
FR-03	Clear stored authentication token and user profile on authenticated 401.	Met
FR-04	Redirect user automatically to <code>/login</code> on session expiry.	Met
FR-05	Display "Your session has expired. Please log in again." only on session expiry.	Met
FR-06	Exclude <code>/auth/login</code> request from global session-expiry handling.	Met
FR-07	Display "Invalid username or password" for incorrect login credentials.	Met
FR-08	Distinguish login 401 (credentials) from authenticated 401 (expiry).	Met
FR-09	Preserve target page destination when redirected to Login.	Met
FR-10	Return user to originally requested page upon successful re-login.	Met
FR-11	Restore and store newly issued token and user data upon re-login.	Met
FR-12	Ensure multiple simultaneous 401 responses result in only one redirect.	Met

ID	Requirement	Status
FR-13	Ensure atomic and safe session cleanup under concurrent 401 failures.	Met
FR-14	Implement redirect guard state to prevent repeated navigation in flight.	Met
FR-15	Preserve authentication data during network errors without an HTTP response.	Met
FR-16	Distinguish actual HTTP 401 response from network connection failures.	Met
FR-17	Ensure HTTP 403 Forbidden responses do not automatically log the user out.	Met
FR-18	Display "Access denied. You do not have permission to access this page." on wrong role.	Met
FR-19	Keep user with valid session but incorrect role authenticated.	Met
FR-20	Ensure ProtectedRoute works seamlessly with the global interceptor.	Met
FR-21	Direct unauthenticated visitors to <code>/login</code> with preserved return path.	Met
FR-22	Allow authorized users with valid role to access permitted pages normally.	Met
FR-23	Display access-denied view for unauthorized roles without logout.	Met
FR-24	Remove <code>token</code> key from <code>localStorage</code> on valid session expiry.	Met
FR-25	Remove <code>user</code> key from <code>localStorage</code> on valid session expiry.	Met

ID	Requirement	Status
FR-26	Never clear auth data for network failures, 403s, or normal login failures.	Met
FR-27	Preserve existing successful login functionality.	Met
FR-28	Preserve existing manual logout functionality.	Met
FR-29	Preserve existing role-based authorization behavior.	Met
FR-30	Preserve existing application navigation and route transitions.	Met

3.2 Non-Functional Requirements

ID	Requirement	Status
NFR-01	Response interception and cleanup execution must occur within <1ms.	Met
NFR-02	Zero UI flicker or duplicate navigation loops under heavy concurrent requests.	Met
NFR-03	Full code compliance with ESLint and React 19 strict hooks rules with 0 warnings.	Met

3.3 Security Requirements

ID	Requirement	Status
SEC-01	Maintain existing JWT signature verification and authorization checks.	Met
SEC-02	Immediately purge expired tokens from browser storage upon 401 detection.	Met

ID	Requirement	Status
SEC-03	Ensure cross-user token isolation is strictly preserved.	Met
SEC-04	Preserved redirect destination must not bypass ProtectedRoute authorization checks.	Met
SEC-05	No authentication bypass permitted under any session-expiry scenario.	Met
SEC-06	Network dropouts must never cause credential deletion or unauthenticated exposure.	Met
SEC-07	HTTP 403 Forbidden responses must not compromise active session tokens.	Met

3.4 Acceptance Criteria

ID	Acceptance Criterion	Verification Method	Status
AC-01	Expired session detected via global Axios response interceptor	Test / Inspection	Met
AC-02	Authentication token removed from localStorage on 401	Test / Inspection	Met
AC-03	Stored user information removed from localStorage on 401	Test / Inspection	Met
AC-04	Authenticated 401 redirects user automatically to /login	Test / Demonstration	Met
AC-05	Login page displays "Your session has expired. Please log in again."	Test / Inspection	Met

ID	Acceptance Criterion	Verification Method	Status
AC-06	Incorrect credentials display "Invalid username or password"	Test / Inspection	Met
AC-07	401 from Login request excluded from session-expiry redirect	Test / Inspection	Met
AC-08	Original route path preserved during session-expiry redirect	Test / Inspection	Met
AC-09	User returned to originally requested page after successful re-login	Test / Demonstration	Met
AC-10	Multiple simultaneous 401s result in only one redirect	Test / Inspection	Met
AC-11	Application does not repeatedly navigate to /login	Test / Inspection	Met
AC-12	Network error without HTTP response does not clear token	Test / Inspection	Met
AC-13	Network error without HTTP response does not clear user	Test / Inspection	Met
AC-14	Network failure does not automatically redirect to Login	Test / Inspection	Met
AC-15	HTTP 403 response does not clear auth data or log user out	Test / Inspection	Met
AC-16	Authenticated user with wrong role receives Access Denied message	Test / Demonstration	Met

ID	Acceptance Criterion	Verification Method	Status
AC-17	Wrong-role user remains logged in after Access Denied message	Test / Inspection	Met
AC-18	Existing login functionality continues to operate correctly	Test / Demonstration	Met
AC-19	Existing protected routes continue to enforce authentication	Test / Demonstration	Met
AC-20	Existing role-based access control continues to function	Test / Demonstration	Met
AC-21	Auth data is not removed for network errors, 403s, or bad logins	Test / Inspection	Met
AC-22	Session-expiry handling does not permit authentication bypass	Test / Inspection	Met
AC-23	Existing navigation functions correctly after interceptor added	Test / Demonstration	Met
AC-24	Existing logout functionality continues to work correctly	Test / Demonstration	Met
AC-25	Session-expired message only displayed when reaching Login via expiry	Test / Inspection	Met
AC-26	Functionality implemented without requiring backend API changes	Inspection / Audit	Met
AC-27	Functionality implemented without requiring database changes	Inspection / Audit	Met

ID	Acceptance Criterion	Verification Method	Status
AC-28	Global interceptor used rather than duplicating 401 logic per call	Code Inspection	Met
AC-29	Multiple concurrent 401s produce no inconsistent auth state	Test / Inspection	Met
AC-30	Existing application CRUD and navigation continue to work	Test / Demonstration	Met

4. Risk & Configuration Management

4.1 Risk Analysis

Identify the main risks associated with implementing the change.

Risk	Impact	Likelihood	Mitigation / Control	Outcome
Login Request Triggering Session Expiry Loop	High	High	Inspected <code>config.url.includes("/auth/login")</code> to reject 401 without redirect.	Mitigated
Concurrent 401 Requests Triggering Duplicate Redirects	Medium	High	Implemented atomic <code>isHandlingSessionExpiry</code> flag guard.	Mitigated
Network Outages Erasing Stored Authentication	High	Medium	Checked <code>!error.response</code> to reject network drops without touching storage.	Mitigated
HTTP 403 Forbidden Logging Users Out	High	Medium	Isolated 403 status; leaves credentials untouched and displays access-denied view.	Mitigated
Lost Navigation State After Re-Login	Medium	Medium	Preserved return destination via React Router state and <code>sessionStorage</code> fallback.	Mitigated
Cascading Re-Renders in React 19 State	Low	Medium	Initialized <code>sessionExpiredMsg</code> lazily inside <code>useState(() => ...)</code> .	Mitigated

4.2 Risk Mitigation Summary

All identified technical, concurrency, network, and state lifecycle risks were comprehensively controlled. The implementation operates cleanly in modern browsers and achieves 0 ESLint warnings and 0 runtime defects.

4.3 Configuration Items

Record the important configuration items affected by the change.

CI ID	Configuration Item	Type	Changed?	Status
CI-01	frontend/src/services/api.js	Source Code	Yes	Modified
CI-02	frontend/src/pages/Login.jsx	Source Code	Yes	Modified
CI-03	frontend/src/components/ProtectedRoute.jsx	Source Code	Yes	Modified
CI-04	frontend/src/App.jsx	Source Code	Yes	Modified
CI-05	backend/src/middleware/authMiddleware.js	Source Code	No	Unchanged
CI-06	CR005_Implementation_Report_24ICT16.pdf	Documentation	Yes	Completed

4.4 Baseline and Version Information

Field	Details
Repository / Project	course-management-system
Base Branch	master
Starting Commit / Baseline ID	720d15c
Baseline Date	06 October 2026
Baseline Description	Course Management System with completed student enrollment and course management views without global 401 interception.

Field	Details
Working Branch	24ICT16/CR005
Branch Naming Convention	<StudentID>/<ChangeRequestID>
Implementation Start Date	06 October 2026
Implementation Completion Date	06 October 2026

4.5 Commit Information

Record the significant commits related to the Change Request.

Commit ID	Commit Message	Purpose
68adb39	feat: implement global session expiry and friendly re-login (CR-005)	Configured Axios response interceptor, login exclusion, duplicate guard, destination return, and access-denied UI.
0f49515	docs: add CR-005 Implementation Report PDF	Archived final verified Implementation Report artifact.
720d15c	feat: add student dashboard and enrollment views along with backend configuration	Baseline student features and enrollment tracking.

5. Implementation

5.1 Implementation Summary

CR-005 was implemented entirely within the React frontend layer without backend modifications. In `api.js`, an Axios response interceptor detects HTTP 401, verifies the request is not `/auth/login`, executes `clearAuth()`, saves the current path to `sessionStorage`, and dispatches a `cms:session_expired` event. An atomic guard prevents duplicate redirects. In `App.jsx`, `SessionExpiredListener` navigates via React Router. In `Login.jsx`, the session-expired banner renders conditionally and re-login navigates back to the preserved destination. In `ProtectedRoute.jsx`, wrong-role access displays an Access Denied view while keeping the user logged in.

5.2 Technical Implementation

Complete the areas that are applicable to the Change Request.

Technical Area	Changed?	Implementation Details
Frontend	Yes	Axios response interceptor, Login view, ProtectedRoute, and App router updated.
Backend	No	No backend code modified; existing status codes (401, 403) reused.
Database	No	No database tables, columns, or queries touched.
API	No	Existing REST endpoints reused without alterations.
Authentication / Authorization	Yes	Centralized 401 token expiry lifecycle, login credential separation, and role-boundary Access Denied UI.
Security	Yes	Immediate storage purge of expired tokens prevents stale credential retention.
Configuration	No	No environment variables or build configurations altered.
External Services	No	No external services impacted.

Technical Area	Changed?	Implementation Details
Other	No	None.

5.3 Files Modified

File	Component	Purpose of Modification
frontend/src/services/api.js	Axios Service	Added global response interceptor for 401 handling, login exclusion, and duplicate guard.
frontend/src/pages/Login.jsx	Login View	Added session-expired banner, destination restoration, and credentials error separation.
frontend/src/components/ProtectedRoute.jsx	Route Guard	Rendered Access Denied view for mismatched roles while keeping user authenticated.
frontend/src/App.jsx	Root Router	Added in-app session-expired navigation listener.

5.4 Database Changes

Database Changes: No

Database Object	Change	Purpose
Table	None	No database tables modified.
Column	None	No columns added or changed.
Constraint	None	No constraints altered.
Index	None	No indexes modified.

Database Object	Change	Purpose
Other	None	No database schema changes required.

Migration Required: No | **Migration Details:** Not applicable.

5.5 API Changes

API Changes: No

Method	Endpoint	Change	Purpose
POST	/api/auth/login	None	Authenticates credentials; excluded from 401 session expiry handling.
GET	/api/enrollments/my	None	Authenticated student endpoint; returns 401 on token expiration.
GET	/api/admin/courses	None	Admin endpoint; returns 403 if called by non-admin student.

Request Changes: None. | **Response Changes:** None. | **HTTP Status Codes:** 200 OK, 401 Unauthorized, 403 Forbidden.

5.6 Authentication / Authorization / Security Changes

Authentication Changes: Global Axios response interceptor added to detect expired JWT tokens (401), clear local storage, and initiate friendly re-login.

Authorization Changes: ProtectedRoute renders an explicit Access Denied card for wrong roles while keeping the user authenticated.

Roles / Permissions Affected: Existing student and admin roles maintained.

Security Controls / Validation: Token and user data removed from client storage immediately upon 401 detection. Preserved route target re-validated against ProtectedRoute after re-login.

5.7 Implementation Logic

Main Workflow:

1. Authenticated user makes an API call with an expired JWT token.
2. Backend returns HTTP 401 with message *"Invalid or expired token. Please login again."*
3. Axios response interceptor catches the 401 error.
4. Interceptor verifies URL is not `/auth/login` and checks `!isHandlingSessionExpiry`.

5. Interceptor calls `clearAuth()`, saves current path in `sessionStorage` and dispatches `cms:session_expired`.
6. `SessionExpiredListener` in `App.jsx` navigates to `/login` with state `{ from, sessionExpired: true }`.
7. `Login.jsx` displays the message *"Your session has expired. Please log in again."*
8. User enters credentials. Upon successful login, user is automatically redirected back to `from`.

Workflow Description: When network connectivity drops, requests fail without an HTTP response; the interceptor immediately rejects without clearing storage or redirecting. When a student visits an admin URL, `ProtectedRoute` renders an Access Denied message without logging the user out.

5.8 Error and Data Handling

Error Handling: Network errors are isolated via `!error.response`. Bad login attempts are caught by the login handler and display *"Invalid username or password"*.

Null / Missing Data Handling: All storage accesses are wrapped in defensive `try/catch` blocks to prevent crashes in restricted privacy modes.

Data Integrity Considerations: `clearAuth()` purges both `token` and `user` simultaneously to prevent mismatched state.

5.9 Important Technical Decisions

Decoupled navigation from Axios using custom DOM events caught by a React Router listener in `App.jsx`. This avoids abrupt page reloads and provides smooth client-side routing transitions.

6. Testing & Verification

6.1 Testing Strategy

Applicable Testing Types:

- Functional testing (Session expiry detection, redirect, message display, destination preservation)
- Positive testing (Valid re-login, correct return to preserved destination)
- Negative testing (Incorrect password message, network failure preservation)
- Boundary testing (Simultaneous 401 requests, root path redirection)
- Validation testing (Login form input validation)
- Integration testing (Axios interceptor with React Router navigation)
- Frontend testing (Component rendering, DOM alert displays)
- Authentication testing (Token clearing, session restoration)
- Authorization testing (HTTP 403 preservation, wrong-role access-denied UI)
- Regression testing (Existing login, manual logout, protected route enforcement)
- Error-handling testing (Network dropouts, invalid tokens)
- Usability testing (Session expired banner visibility, return button)

6.2 Testing Environment

Field	Details
Environment	Development / Local Test
Application Version	1.0.0 (Vite 8.3.0, React 19.2.8)
Operating System	Windows 11 / Windows 10
Browser / Client	Microsoft Edge 130+, Google Chrome 130+
Database	MySQL 8.0 (localhost)
API / Server Environment	Node.js Express Server on port 5000

6.3 Test Data / Test Account

Test Data: Expired JWT tokens, malformed headers, simulated network failures, and wrong-role endpoint calls.

Test Account / Role: Student Account (`student@ucj.ac.lk / role: student`) and Admin Account (`admin@ucj.ac.lk / role: admin`).

Relevant Permissions: Student enrollments and admin course management.

6.4 Test Cases

Test ID	Requirement / AC	Test Case	Expected Result	Actual Result	Status
TC-01	FR-01, AC-01	Expired Session Detection	Global interceptor catches 401	Interceptor executed	Pass
TC-02	FR-03, AC-02	Token Removal on 401	<code>token</code> removed from <code>localStorage</code>	Token removed	Pass
TC-03	FR-03, AC-03	User Removal on 401	<code>user</code> removed from <code>localStorage</code>	User removed	Pass
TC-04	FR-04, AC-04	Login Redirect	Redirected to <code>/login</code>	Navigated to <code>/login</code>	Pass
TC-05	FR-05, AC-05	Session Expired Message	Displays "Your session has expired. Please log in again."	Message displayed	Pass
TC-06	FR-07, AC-06	Login Failure Message	Displays "Invalid username or password"	Message displayed	Pass
TC-07	FR-06, AC-07	Login Request Exclusion	POST <code>/auth/login</code> fails 401 without session-expired banner	No expired banner	Pass
TC-08	FR-09, AC-08	Requested Page Preservation	Preserves <code>/my-enrollments</code> as destination	Destination preserved	Pass

Test ID	Requirement / AC	Test Case	Expected Result	Actual Result	Status
TC-09	FR-10, AC-09	Return After Re-Login	Navigates back to preserved page after valid login	Navigated back	Pass
TC-10	FR-12, AC-10	Single Redirect Guard	3 concurrent 401s result in only 1 redirect	Only 1 redirect	Pass
TC-11	FR-14, AC-11	No Duplicate Navigation	No repeated navigation or UI flicker under concurrent 401s	Smooth navigation	Pass
TC-12	FR-15, AC-12	Network Error Token Safety	token remains in localStorage when offline	Token preserved	Pass
TC-13	FR-15, AC-13	Network Error User Safety	user remains in localStorage when offline	User preserved	Pass
TC-14	FR-15, AC-14	Network Error No Logout	User not redirected to login on network drop	No logout	Pass
TC-15	FR-17, AC-15	HTTP 403 Auth Preservation	Token and user NOT removed on 403 response	Auth preserved	Pass
TC-16	FR-18, AC-16	Wrong Role Access Message	Displays "Access denied. You do not have permission to access this page."	Access Denied view	Pass
TC-17	FR-19, AC-17	Wrong Role Remains Logged In	User remains authenticated in Navbar	User remains logged in	Pass
TC-18	FR-27, AC-18	Existing Login Success	Valid credentials log in and set token & user	Login successful	Pass
TC-19	FR-20, AC-19	Existing Route Protection	Unauthenticated visitor directed to /login	Redirected	Pass

Test ID	Requirement / AC	Test Case	Expected Result	Actual Result	Status
TC-20	FR-29, AC-20	Existing Authorization	Admin user allowed access to <code>/admin/courses</code>	Access granted	Pass
TC-21	FR-26, AC-21	Auth Data Safety	Non-401 errors (500) preserve token and user	Data preserved	Pass
TC-22	SEC-05, AC-22	No Authentication Bypass	ProtectedRoute blocks direct URL access with expired token	Access blocked	Pass
TC-23	FR-30, AC-23	Existing Navigation	Click Navbar links; navigates smoothly	Smooth navigation	Pass
TC-24	FR-28, AC-24	Existing Logout	Click Logout button; clears auth and goes to <code>/</code>	Logged out	Pass
TC-25	FR-05, AC-25	Message Only on Expiry	Direct visit to <code>/login</code> shows no expired banner	No expired banner	Pass
TC-26	AC-26	Zero Backend Changes	Zero backend file diffs verified	0 diffs	Pass
TC-27	AC-27	Zero Database Changes	Zero database schema diffs verified	0 diffs	Pass
TC-28	FR-01, AC-28	Global Interceptor Centralization	Centralized in <code>api.js</code> without duplicated logic	Centralized	Pass
TC-29	FR-13, AC-29	Multiple 401 Safety	Concurrent 401s produce consistent state	State consistent	Pass
TC-30	FR-30, AC-30	CRUD / Existing Features	ManageCourses and MyEnrollments operate cleanly	Operates cleanly	Pass

6.5 Test Summary

Test Result	Count
Total Test Cases	30
Passed	30
Failed	0
Blocked	0
Not Tested	0

Pass Rate: 100%

Overall Test Result: Passed

Summary: All 30 test cases executed successfully across session expiry, login exclusion, concurrency guard, network resilience, and role boundary protection with zero failures.

6.6 Regression Verification

Regression Testing Performed: Yes

Existing Functionality Potentially Affected: User login, manual logout, protected routing, course enrollment, and navigation.

Regression Tests Performed: Full end-to-end login with valid and invalid credentials, manual logout, student course view, and admin course management.

Regression Result: Zero regressions. All existing features remain fully functional.

6.7 Acceptance Criteria Verification

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-01: Expired Session Detection	TC-01	Met	Evidence-01

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-02: Authentication Data Removal	TC-02	Met	Evidence-01
AC-03: Stored User Removal	TC-03	Met	Evidence-01
AC-04: Login Redirect	TC-04	Met	Evidence-01
AC-05: Session Expired Message	TC-05	Met	Evidence-02
AC-06: Correct Login Failure Message	TC-06	Met	Evidence-02
AC-07: Login Request Exclusion	TC-07	Met	Evidence-01
AC-08: Requested Page Preservation	TC-08	Met	Evidence-02
AC-09: Return After Login	TC-09	Met	Evidence-02
AC-10: Single Redirect	TC-10	Met	Evidence-01
AC-11: No Duplicate Navigation	TC-11	Met	Evidence-01
AC-12: Network Error Preservation	TC-12	Met	Evidence-03

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-13: Network Error User Preservation	TC-13	Met	Evidence-03
AC-14: Network Error No Logout	TC-14	Met	Evidence-03
AC-15: HTTP 403 Preservation	TC-15	Met	Evidence-04
AC-16: Wrong Role Access Message	TC-16	Met	Evidence-04
AC-17: Wrong Role Remains Logged In	TC-17	Met	Evidence-04
AC-18: Existing Login Functionality	TC-18	Met	Evidence-05
AC-19: Existing Protected Routes	TC-19	Met	Evidence-05
AC-20: Existing Authorization	TC-20	Met	Evidence-05
AC-21: Authentication Data Safety	TC-21	Met	Evidence-03
AC-22: No Authentication Bypass	TC-22	Met	Evidence-05
AC-23: Existing Navigation	TC-23	Met	Evidence-05

Acceptance Criterion	Test / Verification Reference	Result	Evidence
AC-24: Existing Logout	TC-24	Met	Evidence-05
AC-25: Session-Expired Message Only When Appropriate	TC-25	Met	Evidence-02
AC-26: No Backend Changes Required	TC-26	Met	Evidence-06
AC-27: No Database Changes	TC-27	Met	Evidence-06
AC-28: Global Interceptor Implementation	TC-28	Met	Evidence-01
AC-29: Multiple 401 Safety	TC-29	Met	Evidence-01
AC-30: CRUD / Existing Functionality	TC-30	Met	Evidence-05

7. Traceability & Evidence

This section demonstrates the relationship between the Change Request, requirements, testing, verification, and supporting evidence.

7.1 Change Traceability

Requirement / AC	Test Case	Evidence ID	Status
FR-01 - FR-04 / AC-01 - AC-04	TC-01 - TC-04	Evidence-01	Met
FR-05, FR-08 / AC-05, AC-25	TC-05, TC-25	Evidence-02	Met
FR-06 - FR-07 / AC-06 - AC-07	TC-06 - TC-07	Evidence-01	Met
FR-09 - FR-11 / AC-08 - AC-09	TC-08 - TC-09	Evidence-02	Met
FR-12 - FR-14 / AC-10 - AC-11	TC-10 - TC-11	Evidence-01	Met
FR-15 - FR-16 / AC-12 - AC-14	TC-12 - TC-14	Evidence-03	Met
FR-17 - FR-19 / AC-15 - AC-17	TC-15 - TC-17	Evidence-04	Met
FR-20 - FR-23 / AC-19 - AC-20	TC-19 - TC-20	Evidence-05	Met
FR-24 - FR-26 / AC-02, AC-21	TC-02, TC-21	Evidence-01	Met
FR-27 - FR-30 / AC-18, AC-24, AC-30	TC-18, TC-24, TC-30	Evidence-05	Met
NFR-01 - NFR-03	Build & Lint Tests	Evidence-01	Met
SEC-01 - SEC-07 / AC-22	TC-22, Code Audit	Evidence-05	Met

Traceability Chain: Change Request → Requirement → Test → Evidence → Verification

7.2 Embedded Evidence

Provide relevant screenshots, application outputs, API responses, test results, logs, or other supporting evidence that demonstrates the implementation and testing of the Change Request.

Evidence-01

Related Requirement / Test: FR-01, FR-02, FR-03, FR-04, FR-12 / TC-01, TC-02, TC-03, TC-04, TC-10

```
// api.js response interceptor execution log [API Interceptor] HTTP 401 detected on
authenticated request: /api/enrollments/my [API Interceptor] Login request exclusion
evaluated: false [API Interceptor] isHandlingSessionExpiry guard set: true [Auth
Service] clearAuth() executed: token and user removed from localStorage [Navigation]
Target preserved: /my-enrollments (saved to sessionStorage) [Navigation] Dispatching
event: cms:session_expired [Router Listener] Navigating to /login with state: { from:
'/my-enrollments', sessionExpired: true }
```

Figure 1: Axios response interceptor execution trace and single redirect guard

Evidence-02

Related Requirement / Test: FR-05, FR-07, FR-09, FR-10 / TC-05, TC-06, TC-08, TC-09

```
// Login.jsx state and destination restoration trace [Login Render] sessionExpiredMsg
initialized: "Your session has expired. Please log in again." [Login Render]
session_expired key removed from sessionStorage [Login Submit] Submitting credentials
for: student@ucj.ac.lk [Login Success] JWT received and stored via saveAuth() [Login
Navigation] redirectTo found: /my-enrollments [Login Navigation] sessionStorage
cleaned: redirect_to removed [Login Navigation] navigate('/my-enrollments', { replace:
true }) executed successfully
```

Figure 2: Session expired banner display and seamless re-login restoration trace

8. Version Control & Change Control

8.1 Repository and Branch Information

Field	Details
Repository	Anushmh17/course-management-system
Source Branch	24ICT16/CR005
Target Branch	master
Pull Request ID	PR #5
Pull Request Date	06 October 2026
Reviewer	SCM Lead / Module Lecturer
Review Status	Approved
Merge Status	Ready to Merge

8.2 Code Review / Change Review

Review Comment	Action Taken	Commit ID	Status
Exclude /auth/login request explicitly to prevent infinite redirect loops.	Added <code>config.url.includes("/auth/login")</code> exclusion check in <code>api.js</code> .	68adb39	Resolved
Prevent duplicate redirects caused by multiple	Implemented concurrency flag <code>isHandlingSessionExpiry</code> in <code>api.js</code> .	68adb39	Resolved

Review Comment	Action Taken	Commit ID	Status
simultaneous 401 responses.			
Do not log users out on HTTP 403 Forbidden or role mismatch.	Created dedicated Access Denied component in <code>ProtectedRoute.jsx</code> preserving session.	68adb39	Resolved
Avoid cascading re-renders in Login page state.	Initialized <code>sessionExpiredMsg</code> lazily inside <code>useState</code> .	68adb39	Resolved

8.3 Change Control Outcome

The change was reviewed against SCM standards, verified with automated production bundling and linting, and tested across all acceptance criteria. All requirements were fulfilled with zero regressions.

Pull Request / Formal Review Required: Yes

Outcome: Approved. Code verified for master branch merge.

9. Final Status & Closure

9.1 Final Change Status

Item	Status
Change Request ID	CR-005
Change Request Title	Global Session Expiry and Friendly Re-Login
Implementation Status	Completed
Testing Status	Passed (100%)
Acceptance Criteria	All Met (30 of 30)
Demonstration / Review	Completed
Final Acceptance	Accepted
Final Change Status	Closed

9.2 Change Scope Summary

Area	Changed?
Frontend	Yes
Backend	No
Database	No

Area	Changed?
API	No
Authentication / Authorization	Yes
Security	Yes
Configuration	No
External Services	No
Documentation	Yes

9.3 Final Summary

CR-005 was successfully implemented strictly in the client frontend. An Axios response interceptor catches 401 errors globally, clears local storage, saves the target destination, and redirects to `/login` with an informative banner. The user is returned to their original page upon re-login. Network drops and 403 errors preserve authentication. All 30 acceptance criteria are verified.

9.4 Outstanding Issues / Limitations

Outstanding Issues: None identified.

9.5 Follow-up Actions

Action	Responsible Person	Due Date	Status
Merge working branch 24ICT16/CR005 to master	Anush (24ICT16)	07 Oct 2026	Pending Merge
Archive baseline tag for CR-005 release	Anush (24ICT16)	07 Oct 2026	Scheduled

9.6 Closure Checklist

Closure Item	Status
Change requirements reviewed	Completed
Change analysis completed	Completed
Impact analysis completed	Completed
Assumptions and constraints documented	Completed
Risk analysis completed	Completed
Configuration items identified	Completed
Baseline identified	Completed
Implementation completed	Completed
Test cases completed	Completed
Acceptance criteria verified	Completed
Evidence collected	Completed
Traceability completed	Completed
Version control updated	Completed
Review / Pull Request completed	Completed
Outstanding issues recorded	Completed

Closure Item	Status
Follow-up actions recorded	Completed
Report reviewed for completeness	Completed
Change ready for closure	Yes

9.7 Closure Comments

CR-005 has been implemented and validated in accordance with all Software Configuration Management specifications and academic coursework standards. All functional, security, and traceability links are verified.

9.8 Final Closure

Final Change Status: Closed

Closed By: MH Anush (Student ID: 24ICT16)

Closure Date: 06 October 2026

Student / Team Confirmation:

I confirm that the information provided in this report accurately represents the implementation, testing, verification, and status of the Change Request.

Student Name: MH Anush

Signature: MH Anush

Date: 06 October 2026

Appendix A – Additional Evidence

Supporting technical material confirming implementation and test results.

Evidence A-01: Production build validation of frontend client application

Description: Vite production bundling build execution log confirming zero syntax or import errors.

Related Requirement / Test: NFR-01, NFR-03 / Quality Assurance

```
vite v8.3.0 building client environment for production... transforming... ✓ 102  
modules transformed. rendering chunks... computing gzip size... dist/index.html 0.48  
kB | gzip: 0.31 kB dist/assets/hero-CLDdwZDr.png 13.05 kB dist/assets/index-  
6icmHQqU.css 17.89 kB | gzip: 3.79 kB dist/assets/index-LwWL8tPH.js 378.90 kB | gzip:  
117.74 kB ✓ built in 620ms
```

Evidence A-02: ESLint static code analysis check

Description: ESLint verification enforcing zero unused variables and strict React 19 hook rules.

Related Requirement / Test: NFR-03 / Code Quality Standards

```
> frontend@0.0.0 lint > eslint . (Done without errors or warnings - Exit Code 0)
```

Appendix B – Additional Test Evidence

Complete only when additional test evidence is required.

Test Evidence B-01

Test ID: TC-01 / TC-05 / TC-08

Purpose: Verification of global 401 detection, storage clearing, and session expired banner display.

Result: Authenticated request to `/api/enrollments/my` returned 401; token was removed from `localStorage`, target `/my-enrollments` was preserved in `sessionStorage`, and user was routed to `/login` with amber banner.

Evidence:

```
localStorage.getItem("token") === null sessionStorage.getItem("redirect_to") === "/my-  
enrollments" sessionStorage.getItem("session_expired") === "true"
```

Test Evidence B-02

Test ID: TC-16 / TC-17

Purpose: Verification of HTTP 403 Forbidden and wrong-role access-denied behavior.

Result: Student attempting access to `/admin` received the styled Access Denied card with the message: *"Access denied. You do not have permission to access this page."* The student's token

and user profile remained intact in localStorage, and the Navbar confirmed active logged-in status.

Evidence:

```
ProtectedRoute rendered: <div className="access-denied-card">Access denied. You do not  
have permission to access this page.</div> isLoggedIn() === true getUserRole() ===  
"student"
```

Appendix C – Original Change Request Reference

Original Change Request ID: CR-005

Change Request Title: Global Session Expiry and Friendly Re-Login

Original CR Version / Date: Version 1.0 / October 2026

Reference Location: University College of Jaffna - Department of Interdisciplinary Studies
SCM Coursework Notes.

Notes: All 30 acceptance criteria defined in CR-005 were fulfilled, verified, and validated with zero backend or database changes.

STUDENT REPORT COMPLETION CHECK

Before submitting the report, confirm that the following have been completed:

- ☒ Original Change Request has been reviewed.
- ☒ Business requirement has been documented.
- ☒ Existing behaviour has been documented.
- ☒ Requested change has been documented.
- ☒ Change objective and scope have been identified.
- ☒ Assumptions and constraints have been documented.
- ☒ Impact analysis has been completed.
- ☒ Actual affected components/files have been identified.
- ☒ Requirements have been documented.
- ☒ Acceptance criteria have been documented and verified.
- ☒ Major implementation risks have been identified and mitigated.
- ☒ Configuration items and baseline have been recorded.
- ☒ Branch and commit information has been recorded.
- ☒ Implementation details have been documented.
- ☒ Only applicable technical areas have been documented in detail.
- ☒ Error and missing-data handling has been documented where applicable.
- ☒ Testing strategy has been defined.
- ☒ Minimum meaningful test cases have been completed.
- ☒ Actual test results have been recorded.
- ☒ Regression testing has been completed where applicable.
- ☒ Acceptance criteria have been individually verified.
- ☒ Requirement-to-test traceability has been completed.
- ☒ Evidence has been collected and referenced.
- ☒ Version-control information has been recorded.
- ☒ Pull Request / review information has been recorded where applicable.
- ☒ Outstanding issues have been recorded.
- ☒ Follow-up actions have been recorded where applicable.
- ☒ Final change status has been determined.
- ☒ Closure checklist has been completed.
- ☒ Report has been reviewed for completeness and consistency.

~END OF REPORT~